Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3690624.3709225

RESEARCH-ARTICLE

# From Missteps to Mastery: Enhancing Low-Resource Dense Retrieval through Adaptive Query Generation

**ZHENYU TONG**, University of Chinese Academy of Sciences, Beijing, China

**CHUAN QIN**, Chinese Academy of Sciences, Beijing, Beijing, China

**CHUYU FANG**, Baidu, Inc., Beijing, China

**KAICHUN YAO**, Institute of Software Chinese Academy of Sciences, Beijing, Beijing, China

**XI CHEN**, University of Science and Technology of China, Hefei, Anhui, China

**JINGSHUAI ZHANG**, Baidu, Inc., Beijing, China

**View all**

**Open Access Support** provided by:

**University of Science and Technology of China**

**Baidu, Inc.**

**Chinese Academy of Sciences**

**University of Chinese Academy of Sciences**

**Institute of Software Chinese Academy of Sciences**

**PDF Download**
**3690624.3709225.pdf**
**26 March 2026**
**Total Citations:** 5
**Total Downloads:** 393

**Published:** 20 July 2025

**Citation in BibTeX format**

KDD '25: The 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining
*August 3 - 7, 2025*
*Toronto ON, Canada*

**Conference Sponsors:**
SIGMOD
SIGKDD

# From Missteps to Mastery: Enhancing Low-Resource Dense Retrieval through Adaptive Query Generation

Zhenyu Tong*
University of the Chinese Academy of Sciences
Beijing, China
tongzhenyu123@gmail.com

Chuan Qin*
Computer Network Information Center, Chinese Academy of Sciences
Beijing, China
chuanqin0426@gmail.com

Chuyu Fang
Baidu Inc.
Beijing, China
fangchuyu2022@gmail.com

Kaichun Yao
Institute of Software, Chinese Academy of Sciences
Beijing, China
yaokaichun@outlook.com

Xi Chen
University of Science and Technology of China
Hefei, China
chenxi0401@mail.ustc.edu.cn

Jingshuai Zhang
Baidu Inc.
Beijing, China
zhangjingshuai0@gmail.com

Chen Zhu
University of Science and Technology of China
Hefei, China
zc3930155@gmail.com

Hengshu Zhu†
Computer Network Information Center, Chinese Academy of Sciences
Beijing, China
zhuhengshu@gmail.com

## Abstract

Document retrieval, designed to recall query-relevant documents from expansive collections, is essential for information-seeking tasks, such as web search and open-domain question-answering. Advances in representation learning and pretrained language models (PLMs) have driven a paradigm shift from traditional sparse retrieval methods to more effective dense retrieval approaches, forging enhanced semantic connections between queries and documents and establishing new performance benchmarks. However, reliance on extensive annotated document-query pairs limits their competitiveness in low-resource scenarios. Recent research efforts employing the few-shot capabilities of large language models (LLMs) and prompt engineering for synthetic data generation have emerged as a promising solution. Nonetheless, these approaches are hindered by the generation of lower-quality data within the conventional dense retrieval training process. To this end, in this paper, we introduce **iGFT**, a framework aimed at enhancing low-resource dense retrieval by integrating a three-phase process—**G**eneration, **F**iltering, and **T**uning—coupled with an **i**terative optimization strategy. Specifically, we first employ supervised fine-tuning on limited ground truth data, enabling an LLM to function as the generator capable of producing potential queries from given documents. Subsequently, we present a multi-stage filtering module to minimize noise in the generated data while retaining samples poised to significantly improve the dense retrieval model's performance in the follow-up fine-tuning process. Furthermore, we design a novel iterative optimization strategy that dynamically optimizes the query generator for producing more informative queries, thereby enhancing the efficacy of the entire framework. Finally, extensive experiments conducted on a series of publicly available retrieval benchmark datasets have demonstrated the effectiveness of the proposed iGFT.

## CCS Concepts

• **Information systems** → **Information retrieval**; • **Computing methodologies** → *Natural language generation.*

## Keywords

Dense retrieval; query generation; large language model

**ACM Reference Format:**
Zhenyu Tong, Chuan Qin, Chuyu Fang, Kaichun Yao, Xi Chen, Jingshuai Zhang, Chen Zhu, and Hengshu Zhu. 2025. From Missteps to Mastery: Enhancing Low-Resource Dense Retrieval through Adaptive Query Generation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3690624.3709225

## 1 Introduction

Document retrieval has played a significant role within modern information-seeking systems, aiming to identify the most relevant documents from vast corpora in response to user queries [12, 38, 58]. This foundational process underpins a wide range of applications, from established search engines [25] to the latest retrieval-augmented generation (RAG) frameworks [28].

The field of document retrieval has a rich research history, with traditional approaches predominantly utilizing sparse retrievers, such as TF-IDF and BM25 [43], to match queries and documents through lexical overlap. With the advancement of deep learning,

*Both are co-first authors and contribute equally to this work.
†Corresponding author

This work is licensed under a Creative Commons 4.0 International License.
*KDD '25, August 3–7, 2025, Toronto, ON, Canada*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1245-6/25/08
https://doi.org/10.1145/3690624.3709225

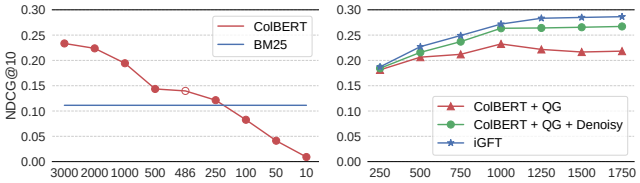


**Figure 1: Left: Declining performance of ColBERT on the FiQA dataset with reducing training data (hollow circle indicates training set used in the SPTAR [37] experimental setup. Right: Impact of fine-tuning the ColBERT model (pretrained using the same training set of SPTAR) with synthetic data generated by LLM-based query generator (QG), QG enhanced with data denoising, and our proposed iGFT across various augmented data volumes.**

recent methods leveraging a dual-encoder architecture have transformed document retrieval by embedding queries and documents into low-dimensional dense vectors for relevance calculation, marking the emergence of dense retrieval [17, 58]. Due to their profound capability to capture intrinsic semantics, pretrained language models (PLMs) like BERT [9, 23] have emerged as the de-facto implementation for encoders in dense retrieval, consequently redefining performance benchmarks in the field [24].

Despite its superior performance over sparse retrieval methods across a broad range of benchmark datasets, dense retrieval heavily relies on a wealth of annotated document-query pairs for effective training and often exhibits poor generalization across domains [49]. This dependency is especially pronounced in low-resource scenarios, where the scarcity of annotated data and the labor-intensive process of collecting well-labeled document-query pairs [41] exacerbate the challenge of improving dense retrieval performance. In response, several researchers have begun constructing pseudo document-query pairs and leveraging contrastive learning techniques to train dense retrieval models, aiming to overcome this obstacle [26]. Recently, large language models (LLMs), including ChatGPT and GPT-4, have demonstrated remarkable capabilities in language understanding, generation, and few-shot learning across various NLP tasks [5, 6, 20]. This advancement introduces a data augmentation perspective for researchers to boost the performance of low-resource dense retrieval by generating potential queries from given documents [4, 19, 37, 44]. For instance, InPars capitalizes on GPT-3's in context learning to generate queries for unlabeled documents [4], while SPTAR employs soft prompt tuning to optimize prompts based on limited ground truth data [37].

However, employing LLMs as query generators to enhance dense retrieval performance still presents several technical challenges. (1) *How can LLMs be enabled to generate high-quality queries?* Current methods, such as InPars [4], InPars-v2 [19], and PROMPTAGATOR [8], primarily rely on prompt engineering to guide LLMs in generating queries, using a small set of positive document-query pairs as examples. These approaches depend heavily on the prompt quality and the LLMs' few-shot learning capabilities, which often result in inconsistent query quality, thereby compromising the effectiveness of subsequent dense retrieval model training. Furthermore, these methods overlook the potential benefits of enhancing the LLM-based query generator by fine-tuning LLMs with annotated document-query pairs. (2) *How can generated queries be filtered to improve the performance of dense retrieval models?* Since LLMs do not always generate high-quality queries, incorporating a data filtering module before training dense retrieval models becomes essential. Previous studies have typically relied on using the generation probability of queries [4] to weed out low-quality data. However, these methods overlook a critical perspective: the identification and selection of the most informative document-query pairs for improving retrieval effectiveness. (3) *How can the generator be optimized to more consistently produce high-quality document-query pairs?* Although LLM-based generators can continuously generate new data, it is important to recognize that not all generated data contributes positively to the performance of dense retrieval models. As illustrated in Figure 1, the marginal utility of data from a static generator in enhancing retrieval model performance diminishes throughout the training process. Therefore, continuously optimizing generators to more reliably produce high-quality synthetic data, particularly to improve the performance of dense retrieval models, has become a critical challenge.

To address the above challenges, in this paper, we introduce a novel framework named **iGFT**, designed to enhance dense retrieval through a three-phase, **i**teratively optimized process—**G**eneration, **F**iltering, and **T**uning. Specifically, we begin by leveraging supervised fine-tuning on limited ground truth data, enabling an LLM to serve as a generator capable of producing potential queries from given documents. Following this, we introduce a multi-stage filtering module designed to reduce noise in the generated data while preserving the most informative samples that can significantly enhance the performance of the dense retrieval model during subsequent fine-tuning. Additionally, we design a novel iterative optimization strategy aimed at augmenting the dense retrieval model's training efficacy. In particular, we present a difficulty-guided reinforcement learning method, dynamically adapting the query generator to produce more informative synthetic data at each iteration. Finally, extensive experiments conducted on several publicly available retrieval benchmark datasets have demonstrated the effectiveness of the proposed framework.

## 2 Related Works

### 2.1 Dense Retrieval

Compared with traditional bag-of-words-based sparse retrieval models [1, 43, 56], neural network-based dense retrieval [22, 24, 39] has the potential to capture deeper semantic information. Early explorations in dense retrieval focused on directly capturing latent semantic characteristics for matching through neural network training [13, 14]. Recently, the advent of pretrained models has significantly enhanced the language understanding ability of the model. A notable example is DPR [22] which employs a BERT-based encoder to independently encode queries and documents, calculating relevance via dot product similarity. Furthermore, MVR [57] introduced the strategy of inserting multiple tokens at the beginning of the text to obtain diverse representations. For more fine-grained vector interaction, ColBERT [24] innovatively generated independent vector representations for each word in queries and documents. In addition,

several studies have proposed learning phrase-level representations of queries when retrieving phrase-based answers [46, 47].

Research has also explored effective negative sampling methods for training retrieval models. For instance, *Xiong et al.* employed the retrieval model trained in the preceding iteration to identify new negative instances for the subsequent training iteration [54]. Additionally, RocketQA [39] enhanced dense retrieval performance by using knowledge distillation to denoise negative samples. Besides, Contriever [18] employed contrastive learning to achieve unsupervised training in dense retrieval.

While dense retrieval demonstrates superior performance, prior research has primarily concentrated on learning from existing query labels. In low-resource scenarios, dense retrieval exhibits discernible limitations [59]. To address this issue, we propose a novel framework comprising a three-phase process: Generation, Filtering, and Tuning, aimed at enhancing low-resource dense retrieval.

## 2.2 Data Augmentation for Dense Retrieval

Leveraging data augmentation techniques to augment the availability of pseudo data can effectively enhance the robustness of the model [2, 42]. In the field of dense retrieval, several studies relied on handcrafted templates and features to extract more relevant paired data [36, 40]. Additionally, some researchers proposed using extraction to select portions of document content as queries or answers to achieve data enhancement [26, 45].

In recent years, researchers have utilized large language models to generate queries for unlabeled corpora, achieving favorable outcomes in information retrieval tasks [4, 8, 11, 33, 34, 44, 52]. InPars utilized prompts with a limited number of examples to generate paragraph-level queries based on GPT-3 [4]. Similarly, TQGen [31] explored query extraction and query generation to create pseudo document-query pairs for augmenting retriever training. In contrast to using the hard prompt for generation, SPTAR [37] introduced soft prompt tuning to optimize the quality of the generated query. Furthermore, to address the inconsistency of generated queries by LLMs, some studies have concentrated on designing various data selection methods to filter out qualified data for training [8, 44].

Beyond training a generator using labeled document-query pairs, some researchers have explored the direct generation of pseudo data based on pretrained models in zero-shot scenarios [15, 30, 48]. Additionally, SPAR [7] employed knowledge distillation from sparse retrieval models to train the data generator in zero-shot settings. Without training the data generator, methods like HyDE [10] and CSQE [27] achieved superior zero-shot information retrieval by supplementing queries or documents with generated information.

However, current LLM-based data generators primarily rely on existing labeled data and powerful pretrained models, lacking the ability to improve autonomously in low-resource scenarios. Moreover, there is a significant deficiency in effective methods for assessing and selecting generated results. In this paper, we propose a multi-stage filtering module designed to minimize noise in the generated data and retain samples most likely to significantly improve the performance of dense retrieval models. Additionally, we introduce a difficulty-guided iterative optimization strategy to continuously enhance the capabilities of data generators.

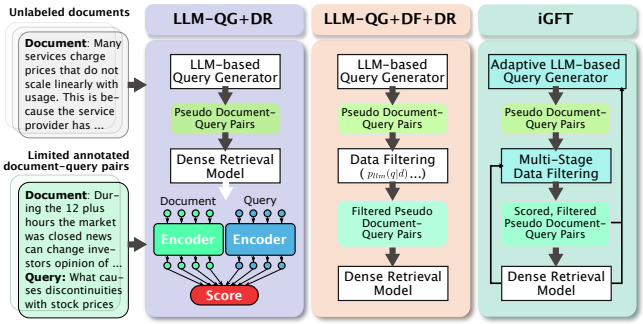


**Figure 2: Enhancing dense retrieval with LLM-based query generation: a comparison of paradigms.**

## 3 Preliminary

Given a large corpus $\mathcal{D} = \{d_i\}_{i=1}^{N}$ composed of $N$ documents, and a natural language query $q$, the objective of document retrieval is to learn a model $\mathcal{R}$ capable of returning a ranked list of the $n$ most relevant documents $\mathcal{D}^q = [d_1^q, d_2^q, ..., d_n^q]$ for the query $q$.

Unlike traditional sparse retrieval methods like BM25, which depend on lexical matching, dense retrieval models employ two learnable functions that map queries and documents to dense vectors. Formally, let $E_Q(\cdot)$ denote the query encoder, which produces a representation $E_Q(q) \in \mathbb{R}^{k_1}$ for each query $q$. Similarly, the document encoder $E_D(\cdot)$ is defined to map a document $d$ to its representation $E_D(d) \in \mathbb{R}^{k_2}$. Along this line, the dense retrieval model $\mathcal{R}$ can be denoted by

$$\mathcal{R}(q, d; \theta) = f_{sim}\left(E_Q(q), E_D(d)\right), \tag{1}$$

where $\theta$ denotes the parameters of $\mathcal{R}$ and the similarity measurement function $f_{sim}(\cdot)$ can be implemented using an inner product, a multilayer perceptron network (MLP), or other neural network architectures. Typically, the representation dimensions of documents and queries are identical, i.e., $k_1 = k_2$. However, this is not always the case. For instance, in ColBERT [24], a well-known dense retrieval model, queries and documents are represented as $E_Q(q) \in \mathbb{R}^{|q| \times k}$ and $E_D(d) \in \mathbb{R}^{|d| \times k}$, respectively, where $k$ represents the token representation dimension. Subsequently, the relevance score is calculated as follows,

$$\mathcal{R}(q, d; \theta) = \sum_{i=1}^{|q|} \max_{j=1,\ldots,|d|} E_Q(q)_i^\top E_D(d)_j, \tag{2}$$

where $|q|$ and $|d|$ denote the number of tokens in $q$ and $d$, respectively. $E_Q(q)_i$ and $E_D(d)_j$ correspond to the embedding vectors for the $i$-th token in $q$ and the $j$-th token in $d$, respectively.

As mentioned before, dense retrieval predominantly employs a supervised training setting. Given a training set $\mathcal{T}_{train}$, where each $(q, d) \in \mathcal{T}_{train}$ represents a relevant document-query pair, the optimization of the dense retrieval model $\mathcal{R}(q, d; \theta)$ can be formalized as:

$$\theta^* = \arg\max_{\theta} \mathbb{E}_{(q,d^+,d^-)\sim\mathcal{T}_{train}} \mathcal{L}(\mathcal{R}, q, d^+, d^-), \tag{3}$$

where $\mathcal{T}'_{train}$ is constructed based on the training set $\mathcal{T}_{train}$. Specifically, $\forall(q, d^+, d^-) \in \mathcal{T}'_{train}$, $d^+$ denotes a positive (relevant) document, i.e., $(q, d^+) \in \mathcal{T}_{train}$, and $d^-$ represents a sampled negative (irrelevant) document, i.e., $d^- \in \mathcal{D}$, $(q, d^-) \notin \mathcal{T}_{train}$. The objective function $\mathcal{L}$ is crucial for model optimization, with the contrastive objective being a prevalent choice. Formally, this is defined as

$$\mathcal{L}(\mathcal{R}, q, d^+, d^-) = \log \frac{exp(\mathcal{R}(q, d^+))}{exp(\mathcal{R}(q, d^+)) + \sum_{j=1}^{l} exp(\mathcal{R}(q, d_j^-))}. \quad (4)$$

In practice, for each query $q$, we sample $l$ documents as negative examples to efficiently train the dense retrieval model $\mathcal{R}$, and we use $d_j^-$ to denote the $j$-th negative documents. Upon completing the training of $\mathcal{R}$, nearest neighbor search tools such as FAISS [21] can be employed to retrieve the top-$n$ most relevant documents for any given query $q$ with sublinear complexity.

As highlighted in the introduction, collecting a sufficient number of document-query pairs for training in low-resource scenarios can be challenging, leading to suboptimal retrieval performance. Moreover, most retrieval datasets contain a vast number of unlabeled documents (i.e., documents not appearing in $\mathcal{T}_{train}$). While dense retrieval models perform well on annotated documents, they may struggle to effectively recall these unlabeled ones. To address this issue, our study introduces a three-phase approach—Generation, Filtering, and Tuning—integrated with an iterative optimization strategy, which significantly enhances the performance of dense retrieval models in low-resource scenarios.

# 4 Methodology

In this section, we elaborate on the details of the iGFT framework, incorporating three processes: LLM-based query generation, multi-stage data filtering, and fine-tuning of the dense retrieval model. Additionally, we introduce a dynamic iterative optimization strategy, specifically designed to refine the query generator. The framework overview is depicted in Figure 3.

## 4.1 LLM-based Query Generation

To enhance the performance of dense retrieval model $\mathcal{R}$ in low-resource settings, recent studies [4, 44] have leveraged LLMs to generate appropriate queries for documents within corpora that lack sufficient annotations. However, the effectiveness of these methods relies heavily on the design of prompts, especially the quality of document-query examples used in in-context learning, which do not guarantee the consistent generation of high-quality queries across varied corpora.

To address this challenge, we utilize Llama-2 [51], an open-source LLM, and apply Supervised Fine-Tuning (SFT) on a constrained dataset of annotated document-query pairs, $\mathcal{T}_{train}$, to develop a query generation model, $\mathcal{G}$. Specifically, as illustrated in the top left corner of Figure 3, we first construct the SFT data based on $\mathcal{T}_{train}$. Subsequently, given the substantial computational resources required for full-parameter fine-tuning, we opt for LoRA [16], a parameter-efficient fine-tuning technique, which retains the core parameters of the LLM unchanged and focuses on training rank decomposition matrices specific to each layer of the Transformer architecture. Along this line, the learning objective for the generator is defined as follows:

$$\mathcal{L}_{llm} = \max_{\Theta_L} \sum_{(q,d)\in\mathcal{T}_{train}} \sum_{t=1}^{|q|} \log \left(p_{\Theta+\Theta_L}(q_t|d, q_{<t})\right), \quad (5)$$

where $\Theta_L$ represents the LoRA parameters, we exclusively update these LoRA parameters throughout the training process. The notation $q_t$ refers to the $t$-th token in $q$, and $q_{<t}$ represents the sequence of tokens $\{q_1, q_1, ..., q_{t-1}\}$. Finally, the fine-tuned LLM $\mathcal{G}$ empowers us to generate queries for any given document, as represented by:

$$\widetilde{q} = \mathcal{G}(d; \Theta + \Theta_L). \quad (6)$$

This capability facilitates the creation of a synthetic dataset $\mathcal{T}_G$ to enhance the performance of dense retrieval model $\mathcal{R}$.

## 4.2 Multi-Stage Data Filtering

Despite the generator $\mathcal{G}$, trained on annotated data, is capable of generating high-quality queries for constructing synthetic dataset $\mathcal{T}_G$, as highlighted in our introduction, $\mathcal{T}_G$ unavoidably includes noise that may undermine the training efficacy of the dense retrieval model $\mathcal{R}$. To mitigate this concern, we develop a multi-stage data filtering module that focuses on both the quality of pseudo data and its utility in training $\mathcal{R}$.

*4.2.1 Filtering with Sparse Retrieval.* Initially, we employ BM25 for filtering the $\mathcal{T}_G$. Specifically, for each pseudo document-query pair $(q_i, d_j) \in \mathcal{T}_G$, we randomly select other $m_1$ documents $\mathcal{D}_m = \{d_k^m\}_{k=1}^{m_1}$ from the corpus $\mathcal{D}$ as negative samples. This process forms a candidate set $\mathcal{D}'_m = \{d_j\} \cup \mathcal{D}_m$. In our experiments, $m_1$ is set to 100. Subsequently, for each $d_k \in \mathcal{D}'_m$, $(q_i, d_k)$ is scored using BM25, denoted as BM25$(q_i, d_k)$. If BM25$(q_i, d_j)$ yields the highest score, we retain this pseudo-pair in $\mathcal{T}_G$; otherwise, it is excluded. By applying this criterion across $\mathcal{T}_G$, we refine it to $\mathcal{T}_G^1$.

*4.2.2 Filtering with Dense Retrieval.* Following the initial filtration with BM25, which eliminates some lower-quality document-query pairs, we recognize the limitation of BM25 to primarily lexical-level relevance. Consequently, we further employ a pretrained dense retrieval model $\mathcal{R}_{pre}$ for a second round of filtering. This subsequent step seeks to capture nuanced semantic relationships that BM25 might miss, ensuring a more discerning selection of training instances for our dense retrieval model $\mathcal{R}$.

Specifically, we first leverage the annotated data $\mathcal{T}_{train}$ to train the pretrained dense retrieval model $\mathcal{R}_{pre}$. In our experiments, $\mathcal{R}_{pre}$ is based on the Colbert model. Subsequently, mirroring the process of filtering with BM25, we construct corresponding negative samples for each query-document pair $(q_i, d_j) \in \mathcal{T}_G^1$. Only those pairs ranked as top-1 based on the score $\mathcal{R}_{pre}(q_i, d_j)$ are selected to form the filtered set $\mathcal{T}_G^2$. Additionally, to further enhance the quality of $\mathcal{T}_G^2$, we sort all query-document pairs in descending order based on the scores $\{\mathcal{R}_{pre}(q_i, d_j), \forall(q_i, d_j) \in \mathcal{T}_G^2\}$. We retain only the top $m_2$% of these pairs to form the new filtered dataset $\mathcal{T}_G^3$. In our experiments, through parameter analysis, we retain 40% of $\mathcal{T}_G^2$ to ensure optimal performance.

*4.2.3 Filtering with Loss Prediction Module.* In the previous stages, our focus was on removing the noisy data. Indeed, selecting samples that offer more informative value to the dense retrieval model $\mathcal{R}$

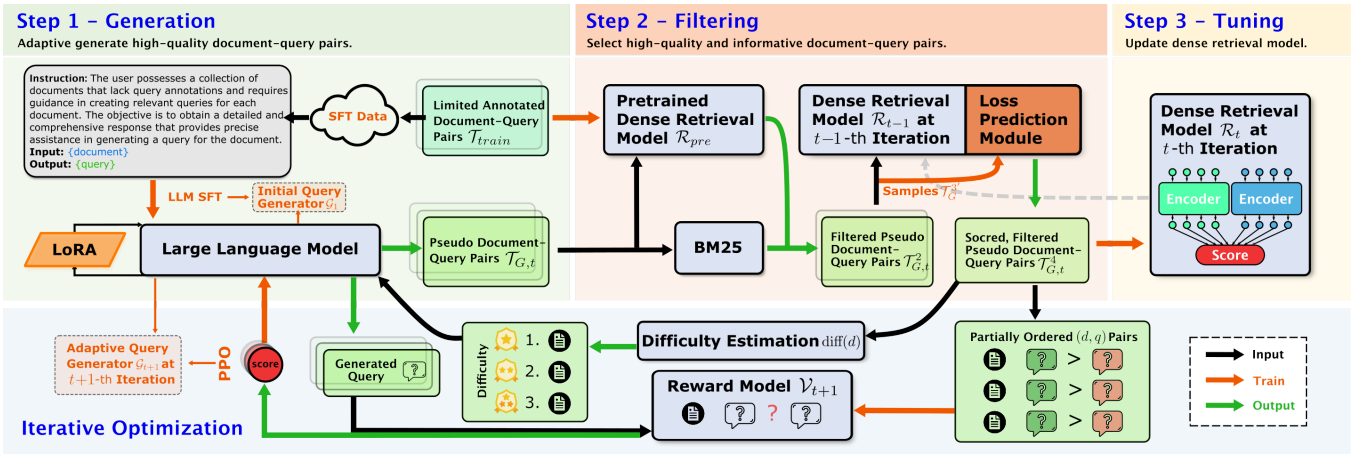


**Figure 3: The overview architecture of our proposed iGFT framework.**

can significantly enhance the training process's effectiveness. To realize this improvement, we introduce a loss prediction module aimed at creating the final filtered dataset.

Specifically, given the filtered data $\mathcal{T}_G^3$, to train dense retrieval model $\mathcal{R}$ more effectively, intuitively, we can select instances from $\mathcal{T}_G^3$ that exhibiting higher loss values:

$$\mathcal{L}(\mathcal{R}, q_i, d_j) = -\mathbb{E}_{d^- \sim \mathcal{D}, (q_i, d^-) \notin \mathcal{T}_G^3} \mathcal{L}(\mathcal{R}, q_i, d_j, d^-), \tag{7}$$

where $(q_i, d_j) \in \mathcal{T}_G^3$ and $\mathcal{L}(\mathcal{R}, q_i, d_j, d^-)$ can be determined by Equation 4. However, due to the high computational cost of sampling negative instances $d^-$, we employ a muti-head cross-attention mechanism to estimate Equation 7. Formally, we have

$$\begin{aligned} h_1 &= MultiHeadAtt\left(E_Q(q_i), E_D(d_j), E_D(d_j)\right), \\ h_2 &= f(h_1), \ y_{q_i,d_j} = mean_pooling(h_2), \end{aligned} \tag{8}$$

where $h_1 \in \mathbb{R}^{|q_i| \times k_3}$, $h_2 \in \mathbb{R}^{|q_i| \times 1}$, $MultiHeadAtt(\cdot, \cdot, \cdot)$ represents the multi-head attention network, $f(\cdot)$ is a learnable multilayer perceptron network, and $E_Q$ and $E_D$ denote the document and query encoders of $\mathcal{R}$, respectively. By uniformly sampling a set of $(q_i, d_j) \in \mathcal{T}_G^3$, denoted as $\mathcal{T}_G^{3'}$, we can estimate the parameters in the loss prediction module by minimizing the following loss:

$$\mathcal{L}_{lp} = \sum_{(q_i, d_j) \in \mathcal{T}_G^{3'}} \left(y_{q_i,d_j} - \mathcal{L}(\mathcal{R}, q_i, d_j)\right)^2. \tag{9}$$

Subsequently, we calculate and sort all $y_{q_i,d_j}$, where $(q_i, d_j) \in \mathcal{T}_G^3$, in descending order. We retain the top $m_3$ of these pairs, which we identify as the most informative instances for optimizing the dense retrieval model, to compose the final filtered dataset $\mathcal{T}_G^4$.

## 4.3 Tuning and Iterative Optimization

*4.3.1 Fine-Tuning Dense Retrieval Model.* As introduced in Section 3, the dense retrieval model can be trained using Equations 3 and 4. Given the annotated training dataset $\mathcal{T}_{train}$, we initially train a dense retrieval $\mathcal{R}_{init}$. Subsequently, leveraging the synthetic data $\mathcal{T}_G^4$ generated by our query generation model $\mathcal{G}$ and refined through a multi-stage filtering process, we further fine-tune $\mathcal{R}_{init}$ to produce our enhanced dense retrieval model $\mathcal{R}$.

*4.3.2 Iterative Optimization Strategy.* While $\mathcal{G}$ can continuously produce new data for training the dense retrieval model $\mathcal{R}$, there is no assurance that $\mathcal{G}$ will consistently generate high-quality and informative data throughout the training process of $\mathcal{R}$. Figure 1 also illustrates that even filtering out low-quality training data, $\mathcal{R}$ rapidly reaches a performance plateau. To address this challenge, we developed an iterative optimization strategy that dynamically updates our generator $\mathcal{G}$. This strategy enables $\mathcal{G}$ to adaptively produce queries that significantly enhance the iterative updates of $\mathcal{R}$, ensuring continuous improvement in retrieval performance.

To clarify our methodology, we begin with the following definitions: $\mathcal{R}_{init}$ represents the initial dense retrieval model trained on $\mathcal{T}_{train}$; $\mathcal{G}_1$, derived from training on the SFT data, is identified as the first iteration of the updated generator; $\mathcal{T}_{G,1}^4$ denotes the synthetic data after multi-stage filtering; and $\mathcal{R}_1$, fine-tuned using $\mathcal{T}_{G,1}^4$ from $\mathcal{R}_{init}$, is recognized as the first iteration of the updated dense retrieval model. Along this line, we denote the generator, synthetic data, and retriever updated in the $t$-th iteration through our iterative optimization strategy as $\mathcal{G}_t$, $\mathcal{T}_{G,t}^4$, and $\mathcal{R}_t$, respectively. Indeed, our primary motivation for iteratively updating the generator is to enable the synthetic data $\mathcal{T}_{G,t}^4$ to include more $(q_i, d_j)$ pairs that challenge $\mathcal{R}_{t-1}$, specifically those with a high $\mathcal{L}(\mathcal{R}_{t-1}, q_i, d_j)$. To this end, we develop a reward model and employ proximal policy optimization (PPO)-based reinforcement learning (RL) to enhance the query generator.

**Reward Model Learning Phase:** The objective of the reward model $\mathcal{V}_t$ at $t$-th iteration is to assign scores to any $(q_i, d_j)$, ensuring that the pairs with a higher $\mathcal{L}(\mathcal{R}_{t-1}, q_i, d_j)$ achieve higher scores. The architecture of $\mathcal{V}_t$ is similar to that of the generator $\mathcal{G}$, but replaces the final output layer with a linear prediction head that outputs scalar reward values. Additionally, the reward model is initialized with the parameters of the generator $\mathcal{G}$. We leverage $\mathcal{T}_{G,t-1}^4$ to construct a dataset comprised of paired comparisons between two responses from $\mathcal{G}_{t-1}$ and employ a pairwise ranking loss to train $\mathcal{V}_t$ in the following manner:

$$\mathcal{L}_{rm} = -\mathbb{E}_{(q_i, q_k, d_j) \sim \mathcal{T}_{rm,t}} \log \sigma(\mathcal{V}_t(q_i, d_j) - \mathcal{V}_t(q_k, d_j)). \tag{10}$$

$\sigma$ denotes the activation sigmoid function. $\mathcal{T}_{rm,t} = \{(q_i, q_k, d_j)\}$, where $(q_i, d_j) \in \mathcal{T}^4_{G,t-1}$, $(q_k, d_j) \in \mathcal{T}^4_{G,t-1}$, and $y_{q_i,d_j} > y_{q_k,d_j}$. $y_{q_i,d_j}$ is calculated by the loss prediction module in Section 4.2.3.

**PPO-based RL Fine-Tuning Phase:** We first use query generator $\mathcal{G}_{t-1}$ and reward model $\mathcal{V}_t$ to initialize the actor model $\mathcal{G}^a$ and critic model $\mathcal{V}^c$. During the RL fine-tuning, we sample the document $d_j \in \mathcal{D}^4_{G,t-1} = \{\forall d_j, s.t. \exists (q_i, d_j) \in \mathcal{T}^4_{G,t-1}\}$ and leverage $\mathcal{G}^a$ to generate query, denoted as $\widetilde{q}_i$. Then, the reward can be formulated as follows:

$$r_{\widetilde{q}_i,d_j} = \mathcal{V}_t(d_j, \widetilde{q}_i) - \lambda \log\left(p(\widetilde{q}_i|d_j, \mathcal{G}^a)/p(\widetilde{q}_i|d_j, \mathcal{G}_{t-1})\right), \quad (11)$$

where $\lambda$ is the coefficient for the KL-divergence term that is used to limit the range of changes in the policy during each update [35]. Meanwhile, the advantage value is the difference between reward $r$ and the value of the input $d_j$ estimated by the critic model as: $a_{\widetilde{q}_{i,k},d_j} = r_{\widetilde{q}_i,d_j} - \mathcal{V}^c(d_j, \widetilde{q}_{i,<k+1})$, where $\widetilde{q}_{i,k}$ denotes the $k$-th token in $\widetilde{q}_i$. Then, we can optimize the actor model $\mathcal{G}^a$ based on:

$$\begin{aligned}\mathcal{L}_{ppo} =& \mathbb{E}_{\substack{\widetilde{q}_i \sim \mathcal{G}^a(d_j) \\ d_j \sim \mathcal{D}^4_{G,t-1}}} \sum_{\widetilde{q}_{i,k} \in \widetilde{q}_i} \min\left(\frac{p(\widetilde{q}_{i,k}|d_j, \widetilde{q}_{i,<k}, \mathcal{G}^a)}{p(\widetilde{q}_{i,k}|d_j, \widetilde{q}_{i,<k}, \mathcal{G}_{t-1})} a_{\widetilde{q}_{i,k},d_j},\right. \\ & \left.\text{clip}\left(\frac{p(\widetilde{q}_{i,k}|d_j, \widetilde{q}_{i,<k}, \mathcal{R}^a)}{p(\widetilde{q}_{i,k}|d_j, \widetilde{q}_{i,<k}, \mathcal{G}_{t-1})}, 1-\varepsilon, 1+\varepsilon\right) a_{\widetilde{q}_{i,k},d_j}\right),\end{aligned} \quad (12)$$

where function $\text{clip}(x, 1-\varepsilon, 1+\varepsilon)$ limits the value of $x$ between $(1-\varepsilon, 1+\varepsilon)$. Finally, the critic model $\mathcal{V}^c$ is optimized with loss function: $\mathcal{L}_c = \mathbb{E}_{(\widetilde{q}_i,d_j)}(r_{\widetilde{q}_i,d_j} - \mathcal{V}^c(d_j, \widetilde{q}_i))^2$.

**Difficulty-Guided Learning Strategy:** By leveraging the reward model and PPO-based RL algorithm, we can update the query generator from $\mathcal{G}_{t-1}$ to $\mathcal{G}_t$. Additionally, to improve the effectiveness of the RL algorithm, we introduce a difficulty-guided learning strategy, so that $\mathcal{G}_t$ can exhibit better performance.

Considering the varying capabilities of generator $\mathcal{G}_{t-1}$ to produce informative queries for different documents, the difficulty faced by the RL process to further enhance $\mathcal{G}_{t-1}$ to generate more informative queries varies across documents. Specifically, given the synthetic data $\mathcal{T}^4_{G,t-1}$, we estimate the difficulty of document $d_j$ for PPO by

$$\text{diff}(d_j) = \frac{1}{|\{(q_i, d_j) \in \mathcal{T}^4_{G,t-1}\}|} \sum_{(q_i,d_j) \in \mathcal{T}^4_{G,t-1}} \frac{rank_{q_i,d_j}}{\mathcal{R}_{t-1}(q_i, d_j)}, \quad (13)$$

where $\mathcal{R}_{t-1}(q_i, d_j)$ represents the relevance score of $(q_i, d_j)$ calculated by the $(t-1)$-th iteration dense retrieval model and $rank_{q_i,d_j}$ denotes the ranking of $d_j$ within all the documents in the corpus $\mathcal{D}$, determined by descending order of $\mathcal{R}_{t-1}(q_i, d_j)$. Indeed, $\text{diff}(d_j)$ effectively estimates the current generator capacity to generate informative for $d_j$. Thus, a higher $\text{diff}(d_j)$ suggests that the generator $\mathcal{G}_{t-1}$ faces greater challenges in further improvement. Along this line, during the PPO-based RL fine-tuning process, instead of using random data shuffling on $\mathcal{D}^4_{G,t-1}$, we sequence the data based on the difficulty level diff from lower to higher for all the $d_j \in \mathcal{D}^4_{G,t-1}$.

# 5 Experiments

## 5.1 Experimental Setting

*5.1.1 Datasets.* To validate the effectiveness of the proposed iGFT in low-resource settings, we selected a series of datasets from the BEIR benchmark [50] to construct our experiments. Specifically, we first selected datasets from BEIR that include document-query training data, such as FiQA, MSMARCO, and NQ. We followed the experimental setup of SPTAR [37], using only 500 document-query pairs as the training set to simulate the low-resource conditions. In addition to the standard low-resource scenarios mentioned above, we followed the BEIR benchmark to validate the model's zero-shot performance. Detailed descriptions of each dataset in BEIR can be found in Appendix A.1.

*5.1.2 Evaluation Metrics.* The evaluation of document retrieval performance relies on assessing the ranking quality, measured through metrics including Mean Average Precision (MAP) [29], Recall [29], Normalized Discounted Cumulated Gains (NDCG) [29], and Mean Reciprocal Rank (MRR) [29]. In this paper, we report the top-10 retrieval performance employing the above metrics—specifically, MAP@10, Recall@10, NDCG@10, and MRR@10.

*5.1.3 Implementation Details.* In the query generation phase of iGFT, we utilized Llama-2 as the generative model. In the SFT stage, the AdamW optimizer with a learning rate of $5e-5$ was utilized. The training spanned across 3 epochs, with a batch size set to 4, incorporating the LoRA [16] technique to achieve parameter efficiency. Please note that in zero-shot scenarios, such as Arguana, due to the lack of document-query training data, we skipped the SFT process for the LLM-based query generation model and instead used the original Llama-2 model as $\mathcal{G}$. Meanwhile, during each iteration of the optimization process, we set the learning rate as $1e-7$ to train the reward model and update the generator. The batch size during this phase was established at 8, and the $\lambda$ in the PPO-based RL fine-tuning stage was fixed at 0.95. In the process of filtering with sparse retrieval, we employed the BM25 algorithm, adjusting the parameters to $b = 0.75$ and $k = 1.5$, setting $m_1 = 100$ to sieve through the synthetic data for quality. Concurrently, for the dense retrieval-based filtering phase, we selected the highest-scoring $m_2 = 40\%$ of $\mathcal{T}^1_G$ as determined by $\mathcal{R}_{pre}$, designating these as meeting the filter's cut-off. Furthermore, within the process of filtering with the loss prediction module, we set $m_3 = 40\%$ to obtain the final filtered data $\mathcal{T}^4_G$. In aligning with the experimental setup of SPTAR, the ColBERT in iGFT was trained using a batch size of 32 and adhered to a learning rate of $2e-5$.

## 5.2 Performance in the Low-Resource Setting

*5.2.1 Baseline Approaches.* To evaluate the effectiveness of iGFT in the low-resource setting, we meticulously selected a range of competitive baseline models for comparison. These include: (1) Sparse retrieval methods: BM25 [43] and its variant BM25-tuned [3], employing a Lucene index for enhanced retrieval efficiency; (2) Dense retrieval model via supervised learning with low resource data: ColBERT [24]; and (3) Dense retrieval model trained via unsupervised learning: ICT [26], MSS [45], and Contriever [18]. Moreover, we incorporated cutting-edge LLM-based query generation methods that aim to enhance dense retrieval models, including Doc2Query [34],

**Table 1: The performances in the low-resource setting of our model and baselines. The order of the top 10 predictions was considered in NDCG, MAP, Recall, and MRR. The best results are in bold, and the second-best results are in underscored.**

| Datasets | FiQA | | | | MSMARCO | | | | NQ | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Metrics** | **NDCG** | **MAP** | **Recall** | **MRR** | **NDCG** | **MAP** | **Recall** | **MRR** | **NDCG** | **MAP** | **Recall** | **MRR** |
| BM25 | 0.1113 | 0.0697 | 0.1913 | 0.1103 | 0.0343 | 0.0151 | 0.1009 | 0.0158 | 0.0789 | 0.0721 | 0.0914 | 0.0800 |
| BM25-tuned | 0.2361 | 0.1784 | 0.2951 | 0.2889 | 0.2084 | 0.1712 | 0.3787 | 0.1733 | 0.2855 | 0.2454 | 0.4555 | 0.2634 |
| ColBERT | 0.1149 | 0.0820 | 0.1547 | 0.1675 | 0.0786 | 0.0619 | 0.1301 | 0.0639 | 0.2560 | 0.2029 | 0.4015 | 0.2225 |
| ICT | 0.1955 | 0.1515 | 0.2278 | 0.2585 | 0.1389 | 0.1095 | 0.2112 | 0.0980 | 0.2601 | 0.1917 | 0.4012 | 0.2077 |
| MSS | 0.1660 | 0.1219 | 0.2167 | 0.2067 | 0.1465 | 0.1180 | 0.2344 | 0.1212 | 0.2414 | 0.1892 | 0.3875 | 0.2047 |
| Contriever | 0.2536 | 0.2002 | 0.2994 | 0.3259 | 0.2056 | 0.1578 | 0.3568 | 0.1611 | 0.2538 | 0.1970 | 0.4128 | 0.2155 |
| Doc2Query | 0.1884 | 0.1392 | 0.2362 | 0.2327 | 0.1827 | 0.1503 | 0.3671 | 0.1702 | 0.2648 | 0.2060 | 0.4673 | 0.2216 |
| Doc2Query-- | 0.2173 | 0.1676 | 0.2712 | 0.2416 | 0.2044 | 0.1758 | 0.3955 | 0.1818 | 0.2829 | 0.2359 | 0.4938 | 0.2529 |
| DocT5Query | 0.2046 | 0.1586 | 0.2385 | 0.2530 | 0.2026 | 0.1717 | 0.3914 | 0.1848 | 0.2767 | 0.2110 | 0.4661 | 0.2264 |
| GPL | 0.2019 | 0.1513 | 0.2335 | 0.2431 | 0.2009 | 0.1751 | 0.3917 | 0.1837 | 0.2641 | 0.2017 | 0.4721 | 0.2134 |
| UDAPDR | 0.1732 | 0.1333 | 0.2185 | 0.2240 | 0.2012 | 0.1392 | 0.2208 | 0.1377 | 0.2620 | 0.2091 | 0.4503 | 0.2170 |
| InPars | 0.2574 | 0.2024 | 0.3051 | 0.3179 | 0.1821 | 0.1444 | 0.2991 | 0.1480 | 0.3097 | 0.2531 | 0.4624 | 0.2701 |
| InPars-v2 | 0.2714 | 0.2158 | 0.3283 | 0.3565 | 0.1890 | 0.1532 | 0.2999 | 0.1566 | 0.3412 | 0.2963 | 0.4892 | 0.3515 |
| SPTAR | 0.2688 | 0.2103 | 0.3083 | 0.3039 | 0.2185 | 0.1872 | 0.3662 | 0.1704 | 0.3007 | 0.2517 | 0.4613 | 0.2607 |
| Promptagator | 0.2351 | 0.1752 | 0.2737 | 0.2725 | 0.2007 | 0.1725 | 0.3927 | 0.1867 | 0.2814 | 0.2315 | 0.4894 | 0.2460 |
| ChatGPT | 0.2697 | 0.2076 | 0.2512 | 0.2507 | 0.2131 | 0.1828 | 0.3987 | 0.1974 | 0.2977 | 0.2417 | 0.5023 | 0.2567 |
| Ours | **0.3042** | **0.2415** | **0.3666** | **0.3731** | **0.2550** | **0.2044** | **0.4108** | **0.2085** | **0.4252** | **0.3898** | **0.5796** | **0.4222** |
| *Improve.* | +12.09% | +11.91% | +11.67% | +4.66% | +16.70% | +9.19% | +3.03% | +5.62% | 24.62% | +31.56% | +15.39% | +20.11% |

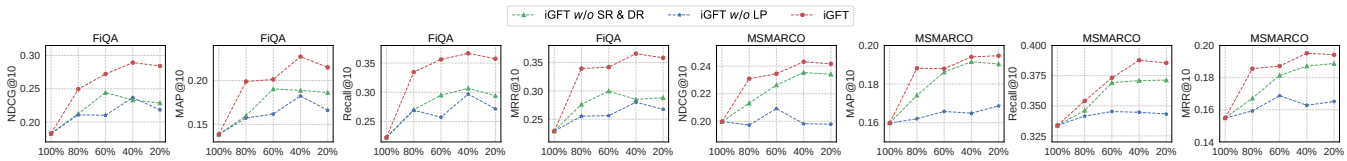


**Figure 4: Comparison of iGFT, iGFT w/o LP, and iGFT w/o SR & DR under various parameter settings.**

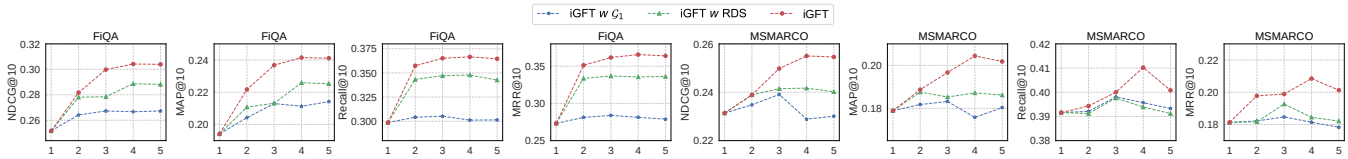


**Figure 5: Comparison of iGFT, iGFT w $\mathcal{G}_1$, and iGFT w RDS at different iterations.**

Doc2Query-- [11], DocT5Query [33], GPL [52], InPars [4], InPars-v2 [19], UDAPDR [44], Promptagator [8], and SPTAR [37]. Furthermore, we utilized ChatGPT for the query generation process for comparative analysis. In our experimental setup, we applied both our iGFT and above LLM-based methods to a pre-trained ColBERT, which ensured a fair and direct comparison of performance.

*5.2.2 Experimental Results.* Table 1 showcases the comparative performance analysis of the proposed iGFT framework against baseline models on the FiQA, MSMARCO, and NQ datasets. We highlighted the best results in boldface and underlined the sub-optimal results. According to the results, there are the following observations: (1) Our iGPT consistently outperforms all baseline models across every dataset, marking significant advancements. Specifically, compared to the best performances of all baselines across various metrics, our iGFT achieves an average improvement of 17.80%, 17.55%, 10.03%, and 10.13% on the NDCG@10, MAP@10, Recall@10, and MRR@10, respectively, across the three datasets. (2) We can observe that ColBERT, relying on supervised learning, does not outperform all sparse retrieval methods. This indicates that dense retrieval models trained solely on annotated data struggle to effectively handle document retrieval tasks in low-resource settings. (3) The unsupervised dense retrieval approaches, including ICT, MSS, and Contriever, which train models by constructing pseudo

**Table 2: The performance comparison of our model and baselines on NDCG@10 in the zero-shot setting. InPars-v2*, LaPraDoR*, and Ours* respectively represent the performances after adding a reranker to the corresponding original dense retrieval model.**

| Models | QGen | LameR | DRAD | InPars-v2 | Query2Doc | HyDE | CSQE | SPAR | LaPraDoR | Ours | InPars-v2* | LaPraDoR* | Ours* |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **ArguAna** | 0.4934 | 0.4021 | 0.4975 | 0.4725 | 0.4146 | 0.4662 | 0.4034 | 0.4591 | 0.5072 | **0.5105** | 0.4690 | 0.5274 | **0.5883** |
| **DBPedia-Entity** | 0.3281 | 0.3957 | 0.4180 | 0.4192 | 0.4247 | 0.3682 | 0.4137 | 0.4275 | 0.4189 | **0.4624** | 0.4982 | 0.4904 | **0.5123** |
| **TREC-Covid** | 0.6083 | 0.7481 | 0.7372 | 0.7184 | 0.7384 | 0.5933 | 0.7422 | 0.7326 | 0.7389 | **0.7712** | 0.8462 | 0.8518 | **0.8794** |
| **FiQA** | 0.3082 | 0.2580 | 0.3382 | 0.3243 | 0.2912 | 0.2768 | 0.2794 | 0.3258 | 0.3290 | **0.3628** | 0.5092 | 0.4973 | **0.5202** |
| **SciFact** | 0.6429 | 0.7251 | 0.6928 | 0.6825 | 0.7172 | 0.6916 | 0.6978 | 0.6962 | 0.6882 | **0.7430** | 0.7742 | 0.7523 | **0.7962** |

**Table 3: The performances in the fully-supervised setting of our model and ColBERT.**

| Datasets | FiQA | | | | MSMARCO | | | | NQ | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Metrics** | **NDCG** | **MAP** | **Recall** | **MRR** | **NDCG** | **MAP** | **Recall** | **MRR** | **NDCG** | **MAP** | **Recall** | **MRR** |
| ColBERT | 0.3312 | 0.2578 | 0.3782 | 0.4193 | 0.3837 | 0.3381 | 0.5127 | 0.3454 | 0.5280 | 0.4169 | 0.6591 | 0.4382 |
| Ours | **0.3572** | **0.2872** | **0.3968** | **0.4312** | **0.4016** | **0.3505** | **0.5680** | **0.3589** | **0.5412** | **0.4557** | **0.7094** | **0.4862** |

$(q, d)$ pairs, utilizing strategies that include segment extraction from documents, have effectively enhanced model performance. Contriever, in particular, outperforms sparse retrieval models across a majority of metrics on the FiQA dataset and achieves competitive performance on the MSMARCO. (4) All enhanced dense retrieval approaches leveraging LLM-based query generation, including our iGFT framework, achieve superior performance compared to previous baseline models. This verifies the capability of LLMs to boost dense retrieval in low-resource scenarios through a data augmentation perspective. Furthermore, our approach outperforms these state-of-the-art LLM-based approaches, thereby demonstrating the effectiveness of our iGFT framework. Furthermore, our experimental results on NFCorpus and SciFact also demonstrate that our iGFT can achieve better performance. Due to space constraints, these results are presented in Appendix A.3.

## 5.3 Analysis of Filtering Strategies

To evaluate the impact of different components within our multi-stage filtering phase, we introduced two variants of our iGFT: (1) **iGFT *w/o* LP**, which excludes the loss prediction module for selecting synthetic data that offer more informative value for training the dense retrieval model, as detailed in Section 4.2.3; (2) **iGFT *w/o* SR & DR**, which removes the processes described in Sections 4.2.1-4.2.2 for filtering out low-quality data via BM25 and $\mathcal{R}_{pre}$.

We reported the performance of our approach and its two variants FiQA and MSMARCO in Figure 4, with the setting of $m_2 = m_3$ ranging from 100% to 20%. As $m_2$ and $m_3$ values increase, the filtered data become more closer to the original synthetic data. Notably, when $m_2 = m_3 = 100\%$, symbolizing a scenario devoid of any filtering processes, BM25 filtering is not implemented. As illustrated in Figure 1, we observed a decline in model performance upon the removal of any component, underscoring the importance of considering both the quality of the generated data and its influence on the dense retrieval model's training within the filtering process. Furthermore, we determined that a configuration of $m_2 = m_3 = 40\%$ yields optimal results. Consequently, we adopted this setting for our iGFT in the experiments.

## 5.4 Analysis of Iterative Optimization

We further investigated the effectiveness of the proposed iterative optimization strategy. Specifically, we introduced two variants of our approach: (1) **iGFT *w* $\mathcal{G}_1$**, which maintains a static generator, i.e., $\mathcal{G}_1$ (trained on the SFT data), throughout the iterative learning process of iGFT; (2) **iGFT *w* RDS**, which adopts a random data shuffle method instead of the proposed difficulty-guided learning strategy for implementing PPO algorithm to update the generator.

Figure 5 presents the performance of our iGFT and its variants at different iterations. We standardized the quantity of data generated per iteration by $\mathcal{G}_t$, ensuring that $|\mathcal{G}_t|$ remains constant. The x-axis denotes the iteration number, with the generator set to $\mathcal{G}_1$ at the first iteration. Results illustrated in Figure 5 reveal that iGFT significantly improves retrieval performance via adaptive updates to the generator throughout the iterative process. Furthermore, the implementation of the proposed difficulty-guided learning strategy in the PPO algorithm substantially elevates iGFT's performance. Furthermore, the time cost analysis of the iterative optimization is presented in Appendix A.2.

## 5.5 Performance in the Zero-Shot Setting

We selected several competitive baseline models known for their effectiveness in zero-shot scenarios, including (1) query-generation methods: QGen [30], LameR [48] and DRAD [15], which focus on generating domain-specific data using pre-trained models without any training data; (2) query expansion methods: Query2Doc [53], HyDE [10] and CSQE [27], which enhance query content by generating additional information to expand the original query; and (3) knowledge distillation methods: SPAR [7] and LaPraDoR [55]. Additionally, we compared the performance of different models after incorporating a reranker. Specifically, we followed InPars-v2 [19], utilizing a pre-trained monoT5-3B model [32] for reranking.

*5.5.1 Experimental Results.* In Table 2, we present a comparative performance analysis of the proposed iGFT method and baseline models on the ArguAna, DBPedia-Entity, TREC-Covid, FiQA, and SciFact datasets. The experimental results show that our method

**Table 4: Case study of query generation based on FiQA dataset.**

| | |
|---|---|
| **Document** $d$ ***(ID:527311)*** | So here's the thing that everyone seems to forget: I bought Netflix to watch MOVIES. Original content is great and all but they started off trying to provide a service and then just abandoned that service to essentially become their own TV network. If I'm bored at home and want to watch A Few Good Men, for example, I can't fire up my Netflix subscription so I'm off to the video store instead, which is exactly the thing I was trying to avoid by subscribing to Netflix. |
| **Generated query** $q$ **(associated with high** $y_{q,d}$**)** | Is Netflix worth it if I never watch streaming movies? |
| **Generated query** $q$ **(associated with low** $y_{q,d}$**)** | What's the problem on Netflix if I want to watch A Few Good Men? |
| **Document** $d$ ***(ID:77792)*** | And that's fine, it's THEIR network that may or may not provide Internet access, they can do what they want with your data (redirect requests or block certain access) while you're using it just as landowners can tell you where you can go and what you can do on their land. Sure, it's shitty, but it's their right to be shitty about it. If you don't want to be subject to that, don't connect to their WiFi network. |
| **Generated query** $q$ **(associated with high** $y_{q,d}$**)** | What are the dangers of connecting to a network? |
| **Generated query** $q$ **(associated with low** $y_{q,d}$**)** | What can a network administrator do to me like a landowner? |

consistently achieves the best performance. Additionally, the models' performance improves to varying degrees with the addition of a reranker. Due to space constraints, the results for the remaining datasets in the BEIR benchmark are provided in Appendix A.3.

## 5.6 Performance in the Fully-Supervised Setting

To further demonstrate the effectiveness of iGFT, we evaluated its performance using fully supervised data. Unlike the previous low-resource experiments, where we sampled training data from FiQA, MSMARCO, and NQ, we used the complete training data to construct $|\mathcal{T}_{train}|$. The experimental results in Table 3 show that even with sufficient training data, our framework still enhances the performance of the dense retrieval model.

## 5.7 Case Study

To enable a more intuitive analysis of the data quality produced by our query generator $\mathcal{G}_t$ for fine-tuning the dense retrieval model, this case study presents examples of the generated queries for the FiQA dataset. As illustrated in Table 4, our query generator $\mathcal{G}_t$ is capable of producing queries tailored to different documents. For instance, document #527311 outlines reasons a user might choose not to renew their Netflix subscription, primarily due to the inability to watch non-Netflix Original content, such as "A Few Good Men". The generated queries demonstrate substantial relevance to the document $d$, further validating the effectiveness of the synthetic data in enhancing the training process of the dense retrieval model. Notably, the first generated query exhibits a higher $y_{d,q}$ value compared to the second. The second query directly targets the keyword "A Few Good Men" from the document, while the first requires a degree of inference to formulate an answer based on the document's content, posing a greater challenge for the retrieval model. Consequently, in our iGFT framework, we implement an iterative optimization strategy to update the generator, encouraging it to produce such challenging instances, thereby significantly improving the performance of the dense retrieval model.

Subsequently, we demonstrated that our iteratively optimized generator more easily produces informative synthetic data for dense retrieval models compared to a static generator. Specifically, we used both the initial generator $\mathcal{G}_1$ and its iteratively updated version, $\mathcal{G}_2$ (after one iteration), to generate 10 queries for each document $d \in \mathcal{D}$. This process produced two corresponding synthetic datasets $\widetilde{\mathcal{T}}_{G,1}$ and $\widetilde{\mathcal{T}}_{G,2}$. After applying our multi-stage filtering process, we derived $\widetilde{\mathcal{T}^4}_{G,1}$ and $\widetilde{\mathcal{T}^4}_{G,2}$. Figure 6 presents the distribution of $\text{diff}_d$, for each $d \in \mathcal{D}$, computed from $\widetilde{\mathcal{T}^4}_{G,1}$ and $\widetilde{\mathcal{T}^4}_{G,2}$, respectively. The results indicate that the iterative training of our generator significantly improves the likelihood of generating high-quality and informative synthetic queries, thereby validating the effectiveness of the iterative optimization strategy in enhancing the overall framework.

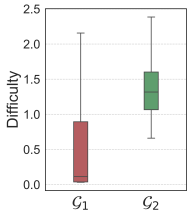


**Figure 6: Distribution of diff$(d)$ calculated from different synthetic data $\widetilde{\mathcal{T}^4}_{G,1}$ and $\widetilde{\mathcal{T}^4}_{G,2}$, which crafted by generators $\mathcal{G}_1$ and $\mathcal{G}_2$, respectively. It can be observed that our iteratively updated generator is more likely to produce samples with higher diff$(d)$.**

## 6 Conclusion

In this paper, we introduced iGFT, a novel framework aimed at enhancing low-resource dense retrieval by integrating a three-phase process—Generation, Filtering, and Tuning—coupled with an iterative optimization strategy. To be more specific, we first employed an LLM to generate appropriate queries for documents with supervised fine-tuning on limited ground truth data. Subsequently, a multi-stage filtering module was present to mitigate noisy data while selecting samples that notably enhance the performance of dense retrieval models. To produce more informative queries, we devised a novel iterative optimization strategy capable of dynamically refining the LLM-based query generator. This strategy facilitated the gradual enhancement of the information retrieval capabilities of the entire framework. Finally, extensive experiments conducted on several publicly available retrieval benchmark datasets have demonstrated the effectiveness of the proposed iGFT.

## Acknowledgments

This work was partially supported by the National Natural Science Foundation of China (No.92470204), the Fundamental Research Project of CNIC (No.E4552304), the Postdoctoral Fellowship Program of CPSF (No.GZC20232811), and the China Postdoctoral Science Foundation (No.2024M753357).

## References


[1] Akiko Aizawa. 2003. An information-theoretic perspective of tf–idf measures. *Information Processing & Management* 39, 1 (2003), 45–65.

[2] Markus Bayer, Marc-André Kaufhold, and Christian Reuter. 2022. A Survey on Data Augmentation for Text Classification. *ACM Comput. Surv.* 55, 7, Article 146 (dec 2022), 39 pages. https://doi.org/10.1145/3544558

[3] Andrzej Białecki, Robert Muir, Grant Ingersoll, and Lucid Imagination. 2012. Apache lucene 4. In *SIGIR 2012 workshop on open source information retrieval*. 17.

[4] Luiz Bonifacio, Hugo Abonizio, Marzieh Fadaee, Rodrigo Nogueira, et al. 2022. InPars: Unsupervised Dataset Generation for Information Retrieval. In *PROCEEDINGS OF THE 45TH INTERNATIONAL ACM SIGIR CONFERENCE ON RESEARCH AND DEVELOPMENT IN INFORMATION RETRIEVAL (SIGIR'22)*. 6.

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[6] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2023. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology* (2023).

[7] Xilun Chen, Kushal Lakhotia, Barlas Oguz, Anchit Gupta, Patrick Lewis, Stan Peshterliev, Yashar Mehdad, Sonal Gupta, and Wen-tau Yih. 2022. Salient Phrase Aware Dense Retrieval: Can a Dense Retriever Imitate a Sparse One?. In *Findings of the Association for Computational Linguistics: EMNLP 2022*. 250–262.

[8] Zhuyun Dai, Vincent Y Zhao, Ji Ma, Yi Luan, Jianmo Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith Hall, and Ming-Wei Chang. 2022. Promptagator: Few-shot Dense Retrieval From 8 Examples. In *The Eleventh International Conference on Learning Representations*.

[9] Chuyu Fang, Chuan Qin, Qi Zhang, Kaichun Yao, Jingshuai Zhang, Hengshu Zhu, Fuzhen Zhuang, and Hui Xiong. 2023. Recruitpro: A pretrained language model with skill-aware prompt learning for intelligent recruitment. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3991–4002.

[10] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise Zero-Shot Dense Retrieval without Relevance Labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1762–1777.

[11] Mitko Gospodinov, Sean MacAvaney, and Craig Macdonald. 2023. Doc2Query–: when less is more. In *European Conference on Information Retrieval*. Springer, 414–422.

[12] Jiafeng Guo, Yinqiong Cai, Yixing Fan, Fei Sun, Ruqing Zhang, and Xueqi Cheng. 2022. Semantic models for the first-stage retrieval: A comprehensive review. *ACM Transactions on Information Systems (TOIS)* 40, 4 (2022), 1–42.

[13] Jiafeng Guo, Yixing Fan, Qingyao Ai, and W Bruce Croft. 2016. A deep relevance matching model for ad-hoc retrieval. In *Proceedings of the 25th ACM international on conference on information and knowledge management*. 55–64.

[14] Jiafeng Guo, Yixing Fan, Liang Pang, Liu Yang, Qingyao Ai, Hamed Zamani, Chen Wu, W Bruce Croft, and Xueqi Cheng. 2020. A deep look into neural ranking models for information retrieval. *Information Processing & Management* 57, 6 (2020), 102067.

[15] Helia Hashemi, Yong Zhuang, Sachith Sri Ram Kothur, Srivas Prasad, Edgar Meij, and W Bruce Croft. 2023. Dense retrieval adaptation using target domain description. In *Proceedings of the 2023 ACM SIGIR International Conference on Theory of Information Retrieval*.

[16] Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2021. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.

[17] Po-Sen Huang, Xiaodong He, Jianfeng Gao, Li Deng, Alex Acero, and Larry Heck. 2013. Learning deep structured semantic models for web search using clickthrough data. In *Proceedings of the 22nd ACM international conference on Information & Knowledge Management*. 2333–2338.

[18] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2022. Unsupervised Dense Information Retrieval with Contrastive Learning. *Transactions on Machine Learning Research* (2022).

[19] Vitor Jeronymo, Luiz Bonifacio, Hugo Abonizio, Marzieh Fadaee, Roberto Lotufo, Jakub Zavrel, and Rodrigo Nogueira. 2023. InPars-v2: Large Language Models as Efficient Dataset Generators for Information Retrieval. *arXiv preprint arXiv:2301.01820* (2023).

[20] Feihu Jiang, Chuan Qin, Kaichun Yao, Chuyu Fang, Fuzhen Zhuang, Hengshu Zhu, and Hui Xiong. 2024. Enhancing question answering for enterprise knowledge bases using large language models. In *International Conference on Database Systems for Advanced Applications*. Springer, 273–290.

[21] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

[22] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics.

[23] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*. 4171–4186.

[24] Omar Khattab and Matei Zaharia. 2020. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 39–48.

[25] Mei Kobayashi and Koichi Takeda. 2000. Information retrieval on the web. *ACM computing surveys (CSUR)* 32, 2 (2000), 144–173.

[26] Kenton Lee, Ming-Wei Chang, and Kristina Toutanova. 2019. Latent Retrieval for Weakly Supervised Open Domain Question Answering. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 6086–6096.

[27] Yibin Lei, Yu Cao, Tianyi Zhou, Tao Shen, and Andrew Yates. 2024. Corpus-Steered Query Expansion with Large Language Models. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*. 393–401.

[28] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

[29] Fangyuan Luo, Jun Wu, and Tao Wang. 2023. Discrete Listwise Content-aware Recommendation. *ACM Trans. Knowl. Discov. Data* 18, 1, Article 7 (aug 2023), 20 pages. https://doi.org/10.1145/3609334

[30] Ji Ma, Ivan Korotkov, Yinfei Yang, Keith Hall, and Ryan McDonald. 2021. Zero-shot Neural Passage Retrieval via Domain-targeted Synthetic Question Generation. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*.

[31] Rui Meng, Ye Liu, Semih Yavuz, Divyansh Agarwal, Lifu Tu, Ning Yu, Jianguo Zhang, Meghana Bhat, and Yingbo Zhou. 2022. Unsupervised Dense Retrieval Deserves Better Positive Pairs: Scalable Augmentation with Query Extraction and Generation. *arXiv preprint arXiv:2212.08841* (2022).

[32] Rodrigo Nogueira, Zhiying Jiang, Ronak Pradeep, and Jimmy Lin. 2020. Document Ranking with a Pretrained Sequence-to-Sequence Model. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 708–718.

[33] Rodrigo Nogueira, Jimmy Lin, and AI Epistemic. 2019. From doc2query to docTTTTTquery. *Online preprint* 6, 2 (2019).

[34] Rodrigo Nogueira, Wei Yang, Jimmy Lin, and Kyunghyun Cho. 2019. Document expansion by query prediction. *arXiv preprint arXiv:1904.08375* (2019).

[35] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems* 35 (2022), 27730–27744.

[36] Shivank Pandey and KC Rajeswari. 2013. Automatic question generation using software agents for technical institutions. *International Journal of Advanced Computer Research* 3, 4 (2013), 307.

[37] Zhiyuan Peng, Xuyang Wu, and Yi Fang. 2023. Soft prompt tuning for augmenting dense retrieval with large language models. *arXiv preprint arXiv:2307.08303* (2023).

[38] Chuan Qin, Le Zhang, Yihang Cheng, Rui Zha, Dazhong Shen, Qi Zhang, Xi Chen, Ying Sun, Chen Zhu, Hengshu Zhu, et al. 2023. A comprehensive survey of artificial intelligence techniques for talent analytics. *arXiv preprint arXiv:2307.03195* (2023).

[39] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Xin Zhao, Daxiang Dong, Hua Wu, and Haifeng Wang. 2021. RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 5835–5847.

[40] Sheetal Rakangor and YR Ghodasara. 2015. Literature review of automatic question generation systems. *International journal of scientific and research publications* 5, 1 (2015), 1–5.

[41] Ori Ram, Gal Shachaf, Omer Levy, Jonathan Berant, and Amir Globerson. 2022. Learning to Retrieve Passages without Supervision. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2687–2700.

[42] Sylvestre-Alvise Rebuffi, Sven Gowal, Dan Andrei Calian, Florian Stimberg, Olivia Wiles, and Timothy A Mann. 2021. Data Augmentation Can Improve Robustness. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 29935–29948. https://proceedings.neurips.cc/paper_files/paper/2021/file/fb4c48608ce8825b558ccf07169a3421-Paper.pdf

[43] Stephen E Robertson, Steve Walker, Susan Jones, Micheline M Hancock-Beaulieu, Mike Gatford, et al. 1995. Okapi at TREC-3. *Nist Special Publication Sp* 109 (1995), 109.

[44] Jon Saad-Falcon, Omar Khattab, Keshav Santhanam, Radu Florian, Martin Franz, Salim Roukos, Avirup Sil, Md Arafat Sultan, and Christopher Potts. 2023. UDAPDR: Unsupervised Domain Adaptation via LLM Prompting and Distillation

of Rerankers. *arXiv preprint arXiv:2303.00807* (2023).
[45] Devendra Sachan, Mostofa Patwary, Mohammad Shoeybi, Neel Kant, Wei Ping, William L Hamilton, and Bryan Catanzaro. 2021. End-to-End Training of Neural Retrievers for Open-Domain Question Answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 6648–6662.
[46] Minjoon Seo, Tom Kwiatkowski, Ankur Parikh, Ali Farhadi, and Hannaneh Hajishirzi. 2018. Phrase-Indexed Question Answering: A New Challenge for Scalable Document Comprehension. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 559–564.
[47] Minjoon Seo, Jinhyuk Lee, Tom Kwiatkowski, Ankur Parikh, Ali Farhadi, and Hannaneh Hajishirzi. 2019. Real-Time Open-Domain Question Answering with Dense-Sparse Phrase Index. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 4430–4441.
[48] Tao Shen, Guodong Long, Xiubo Geng, Chongyang Tao, Tianyi Zhou, and Daxin Jiang. 2023. Large language models are strong zero-shot retriever. *arXiv preprint arXiv:2304.14233* (2023).
[49] Xiaoyu Shen, Svitlana Vakulenko, Marco Del Tredici, Gianni Barlacchi, Bill Byrne, and Adrià de Gispert. 2022. Low-resource dense retrieval for open-domain question answering: A comprehensive survey. *arXiv preprint arXiv:2208.03197* (2022).
[50] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.
[51] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
[52] Kexin Wang, Nandan Thakur, Nils Reimers, and Iryna Gurevych. 2022. GPL: Generative Pseudo Labeling for Unsupervised Domain Adaptation of Dense Retrieval. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.
[53] Liang Wang, Nan Yang, and Furu Wei. 2023. Query2doc: Query Expansion with Large Language Models. In *The 2023 Conference on Empirical Methods in Natural Language Processing*.
[54] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. 2020. Approximate nearest neighbor negative contrastive learning for dense text retrieval. *arXiv preprint arXiv:2007.00808* (2020).
[55] Canwen Xu, Daya Guo, Nan Duan, and Julian McAuley. 2022. LaPraDoR: Unsupervised Pretrained Dense Retriever for Zero-Shot Text Retrieval. In *Findings of the Association for Computational Linguistics: ACL 2022*. 3557–3569.
[56] Peilin Yang, Hui Fang, and Jimmy Lin. 2017. Anserini: Enabling the use of lucene for information retrieval research. In *Proceedings of the 40th international ACM SIGIR conference on research and development in information retrieval*. 1253–1256.
[57] Shunyu Zhang, Yaobo Liang, Ming Gong, Daxin Jiang, and Nan Duan. 2022. Multi-View Document Representation Learning for Open-Domain Dense Retrieval. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 5990–6000.
[58] Wayne Xin Zhao, Jing Liu, Ruiyang Ren, and Ji-Rong Wen. 2022. Dense text retrieval based on pretrained language models: A survey. *arXiv preprint arXiv:2211.14876* (2022).
[59] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Zhicheng Dou, and Ji-Rong Wen. 2023. Large language models for information retrieval: A survey. *arXiv preprint arXiv:2308.07107* (2023).

# A Appendix

## A.1 Dataset Descriptions

We conducted our experiments using a series of datasets included in the BEIR benchmark. The detailed descriptions are provided below:

- **FiQA**: FiQA dataset concentrates on question-and-answer sessions related to financial matters, encompassing a diverse range of inquiries and responses sourced from financial forums.
- **MSMARCO**: MSMARCO is a substantial question-answer and information retrieval dataset. It aims to foster advancements in machine reading comprehension and search engine algorithms.
- **NQ**: NQ centers primarily on question-answering tasks. Each question is grounded in real-world information requirements and correlated with answers extracted from complete web pages.
- **HotpotQA**: HotpotQA is a specialized dataset for studying multi-hop questions in natural language question answering, featuring comprehensive supervision of supporting facts.
- **Quora**: Quora is sourced from the Quora Q&A platform, with questions and answers created by its users. It covers a wide range of topics and includes annotations indicating whether each question is semantically similar to existing questions.
- **CQADupstack**: CQADupStack is a community question-answering dataset sourced from StackExchange, comprising Q&A posts across 12 subdomains, including programming (Stack Overflow), mathematics (Math Stack Exchange), and physics (Physics Stack Exchange).
- **TREC-COVID**: The TREC-COVID dataset is specifically designed for the retrieval of information related to the COVID-19 pandemic. It aims to assist researchers in accessing reliable data about the virus and its impacts.
- **NFCorpus**: NFCorpus is designed for medical information retrieval. It comprises non-technical natural language queries and corresponding complex, terminology-heavy documents.
- **Trec-News**: TREC-News is a dataset for news information retrieval, aimed at enhancing the understanding of content relevance in news retrieval.
- **Robust04**: Robust04 consists of news articles and other texts, focusing on poorly performing topics to advance retrieval techniques.
- **ArguAna**: ArguAna dataset is sourced from debate websites and forums, comprising a large number of speculative questions and responses with supporting and opposing arguments.
- **Touche-2020**: The Touche-2020 dataset comprises contentious question-answer pairs in 2020, where each query includes multiple responses with supporting or opposing positions and arguments.
- **DBPedia-Entity**: DBPedia is extracted from Wikipedia in a structured manner, containing a large number of entities along with their corresponding attributes and relationships.
- **SciDocs**: SciDocs is composed of research papers from a wide range of academic fields, including structured information such as authors, titles, and abstracts. It aims to contribute to advancements in paper information retrieval.
- **Fever**: Fever is a fact verification dataset sourced from Wikipedia. It consists of queries requiring validation, corresponding evidence, and labels indicating the truthfulness of the claims. In the field of information retrieval, it is treated as a task of retrieving evidence corresponding to the queries.
- **Climate-Fever**: Climate-Fever is a fact verification dataset in the climate domain, sourced from scientific papers, news articles, government reports, and more. Similar to Fever, it includes climate-related queries, corresponding evidence, and truthfulness labels.
- **SciFact**: SciFact is a fact verification dataset sourced from peer-reviewed scientific papers. It includes science-related queries, corresponding evidence, and truthfulness labels.

Please note that in our experiments, the BioASQ and Signal datasets from BEIR were not included. This is because these two datasets are not publicly available, and we have not yet succeeded in obtaining access to them.

**Table S1: The performances in the zero-shot setting of our model and representative baselines on all accessible datasets in the BEIR benchmark.**

| **Models** | QGen | InPars-v2 | LaPraDoR | Ours | InPars-v2* | LaPraDoR* | Ours* |
|---|---|---|---|---|---|---|---|
| FiQA | 0.3082 | 0.3243 | 0.3290 | 0.3628 | 0.5085 | 0.4973 | 0.5202 |
| NQ | 0.3583 | 0.4532 | 0.4872 | 0.5221 | 0.6382 | 0.7072 | 0.7119 |
| HotPotQA | 0.5245 | 0.5406 | 0.6241 | 0.6665 | 0.7912 | 0.7832 | 0.8215 |
| Quora | 0.8129 | 0.8080 | 0.8692 | 0.8468 | 0.8451 | 0.8946 | 0.8889 |
| CQADupstack | 0.3589 | 0.3019 | 0.2427 | 0.3792 | 0.4483 | 0.4672 | 0.4925 |
| TREC-COVID | 0.6083 | 0.7184 | 0.7389 | 0.7712 | 0.8462 | 0.8518 | 0.8792 |
| NFCorpus | 0.3032 | 0.3341 | 0.3246 | 0.3815 | 0.3845 | 0.4409 | 0.4252 |
| Trec-News | 0.3872 | 0.3825 | 0.4481 | 0.4323 | 0.4902 | 0.5241 | 0.5135 |
| Robust04 | 0.3567 | 0.4285 | 0.4901 | 0.4805 | 0.6322 | 0.6018 | 0.6312 |
| ArguAna | 0.4934 | 0.4725 | 0.5072 | 0.5105 | 0.4690 | 0.5273 | 0.5583 |
| Touche-2020 | 0.1822 | 0.2845 | 0.3241 | 0.3107 | 0.2905 | 0.3305 | 0.3175 |
| DBPedia-Entity | 0.3281 | 0.4192 | 0.4189 | 0.4624 | 0.4979 | 0.4902 | 0.5123 |
| SciDocs | 0.1429 | 0.1129 | 0.1829 | 0.1675 | 0.2083 | 0.2472 | 0.2238 |
| Fever | 0.6693 | 0.6671 | 0.6821 | 0.7894 | 0.8715 | 0.7894 | 0.8714 |
| Climate-Fever | 0.1755 | 0.1725 | 0.2267 | 0.1947 | 0.3234 | 0.3371 | 0.3451 |
| SciFact | 0.6429 | 0.6825 | 0.6882 | 0.7430 | 0.7743 | 0.7523 | 0.7962 |
| Avg. | 0.3913 | 0.4178 | 0.4461 | 0.4718 | 0.5637 | 0.5776 | 0.5943 |

**Table S2: The efficiency of static generator and iterative updated generator.**

| **Models** | **NDCG** | **MAP** | **Recall** | **MRR** | **Time** | **FLOPS** |
|---|---|---|---|---|---|---|
| Static generator | 0.3262 | 0.2568 | 0.5225 | 0.2826 | 1.6h | 24.32 PFLOP |
| Iterative updated generator | 0.3312 | 0.2583 | 0.5273 | 0.2925 | 2.1h | 34.13 PFLOP |

**Table S3: The performance of static generator and iterative updated generator over iterations.**

| **Models** | **Static generator** | | | |
|---|---|---|---|---|
| **#Iteration** | **NDCG** | **MAP** | **Recall** | **MRR** |
| 1 | 0.3281 | 0.2402 | 0.4817 | 0.3491 |
| 2 | 0.3581 | 0.2517 | 0.4991 | 0.3518 |
| 3 | 0.3579 | 0.2502 | 0.4969 | 0.3512 |
| **Models** | **Iterative updated generator** | | | |
| **#Iteration** | **NDCG** | **MAP** | **Recall** | **MRR** |
| 1 | 0.3672 | 0.2549 | 0.5117 | 0.3682 |
| 2 | 0.4002 | 0.3434 | 0.5310 | 0.3940 |
| 3 | 0.4218 | 0.3842 | 0.5720 | 0.4182 |

## A.2 The Time Cost of the Iterative Optimization

We conducted a detailed analysis of the iterative efficiency during the training process on the FiQA dataset. As illustrated in Table S2, while maintaining an equivalent total output of pseudo queries, we compared the training duration for the reinforcement learning (RL) process with the time and flops (floating point operations) required for each round of direct generation. Our findings reveal that generating an equivalent number of queries using the iterative method requires 31.25% more time and 40.33% more flops. Considering the additional training overhead of RL, coupled with the fact that the iterative updated generator is more adept at producing high-quality data, our proposed method is demonstrated to be more efficient. Moreover, we have conducted an additional experiment to illustrate the performance differences between the models when allocated the same time budget per iteration in Table S3. This highlights that our iterative optimization progressively unlocks the latent potential of the generator, enabling more efficient improvements in the performance of the dense retrieval model while maintaining the same computational resource constraints. In contrast, the static generator faces a performance bottleneck, limiting its effectiveness.

Compared to other query generation models, which do not use iterative computations, our model is more computationally efficient in the non-iterative setting, with InPars [4], Promptagator [8], and UDAPDR [44] taking 2.03h, 2.5h, and 1.92h, respectively, while our model requires only 1.6h. With iterative computations, our model takes 2.1h, 9.38% more than the non-iterative approach. However, as shown in previous experiments, this additional cost is justified by the significant performance gains in dense retrieval achieved through iteration.

## A.3 Addtional Evaluation in the Zero-Shot Setting

Following the experimental setup in Section 5.5, we compared our method with several representative baselines on all accessible datasets in the BEIR benchmark. The results are presented in Table S1. Specifically, we found that our approach achieved an average improvement of 20.57%, 12.92%, and 5.76% in NDCG@10 across all accessible datasets in the BEIR Benchmark, compared to QGen, Inpars-v2, and LaPraDoR, respectively. After incorporating a reranker, our approach continues to achieve improvements of 5.43% and 2.89% for InPars-v2 and LaPraDoR, respectively. These experimental results provide strong evidence that our proposed iGFT method delivers superior performance in the vast majority of zero-shot scenarios.